

```

// api/get-all-entities.js
// Returns connection status + balances for all SOJ entities
// Used by dashboard home to show which entities are connected

const { PlaidApi, PlaidEnvironments, Configuration } = require('plaid');
const { createClient } = require('@supabase/supabase-js');

const plaidConfig = new Configuration({
  basePath: PlaidEnvironments[process.env.PLAID_ENV || 'sandbox'],
  baseOptions: {
    headers: {
      'PLAID-CLIENT-ID': process.env.PLAID_CLIENT_ID,
      'PLAID-SECRET': process.env.PLAID_SECRET,
    },
  },
});

const plaidClient = new PlaidApi(plaidConfig);
const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SERVICE_KEY
);

// Your three business entities
const ENTITIES = [
  { name: 'The Stick of Joseph LLC', short: 'SOJ', color: '#C9963A' },
  { name: 'Plain and Precious Publishing LLC', short: 'PPP', color: '#1E6B45' },
  { name: 'Saint Threads LLC', short: 'ST', color: '#1A4A8A' },
];

module.exports = async (req, res) => {
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'GET, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type, Authorization');

  if (req.method === 'OPTIONS') return res.status(200).end();

  try {
    // — Admins only: this endpoint returns every entity's data —
    const token = (req.headers.authorization || '').replace(/^Bearer\s+/i, '');
    if (!token) return res.status(401).json({ error: 'Not signed in' });
    const { data: userData, error: authErr } = await supabase.auth.getUser(token)
    if (authErr || !userData || !userData.user) return res.status(401).json({ err
    const { data: prof } = await supabase

```

```

    .from('profiles').select('role').eq('email', userData.user.email).maybeSingle
    if (!prof || prof.role !== 'admin') return res.status(403).json({ error: 'Admin access required' });

    // Get all connected items from Supabase
    const { data: items, error: dbError } = await supabase
      .from('plaid_items')
      .select('entity_name, institution_name, last_synced, is_active, account_count')
      .eq('is_active', true);

    if (dbError) throw new Error(dbError.message);

    // For each connected entity, fetch current balances
    const entityResults = await Promise.all(
      ENTITIES.map(async (entity) => {
        const item = items?.find(i => i.entity_name === entity.name);

        if (!item) {
          return {
            ...entity,
            connected: false,
            balance: null,
            last_synced: null,
            institution: null,
          };
        }

        // Get access token
        const { data: fullItem } = await supabase
          .from('plaid_items')
          .select('access_token')
          .eq('entity_name', entity.name)
          .single();

        try {
          const balanceRes = await plaidClient.accountsBalanceGet({
            access_token: fullItem.access_token,
          });

          const totalBalance = balanceRes.data.accounts
            .reduce((sum, a) => sum + (a.balances.current || 0), 0);

          return {
            ...entity,
            connected: true,
            balance: totalBalance,
            last_synced: item.last_synced,
            institution: item.institution_name,
          };
        } catch (error) {
          return {
            ...entity,
            connected: false,
            balance: null,
            last_synced: null,
            institution: null,
          };
        }
      })
    );
  }
}

```

```

        account_count: item.account_count,
    };
} catch (balanceError) {
    // Token may need refresh
    return {
        ...entity,
        connected: true,
        needs_refresh: true,
        balance: null,
        last_synced: item.last_synced,
        institution: item.institution_name,
    };
}
}))
);

const totalConnected = entityResults.filter(e => e.connected).length;
const totalBalance = entityResults
    .filter(e => e.balance !== null)
    .reduce((sum, e) => sum + e.balance, 0);

res.status(200).json({
    entities: entityResults,
    summary: {
        total_connected: totalConnected,
        total_entities: ENTITIES.length,
        total_balance: totalBalance,
        last_updated: new Date().toISOString(),
    },
});

} catch (error) {
    console.error('get-all-entities error:', error.message);
    res.status(500).json({ error: error.message });
}
};

```